

Sprint 9 — Decorator Pattern

Class 2

Learning Objectives

By the end of this class, we should be able to:

- Compare Decorator with Adapter, Proxy, Facade, and Strategy.
- Recognize Decorator in Java I/O.
- Apply Decorator in Spring Boot.
- Understand decorator ordering.
- Test decorated objects correctly.
- Combine Decorator with Adapter and Factory.
- Know when AOP is better than Decorator.
- Avoid overdecorating simple code.

Lesson 22 — Decorator vs Adapter

This is one of the most common interview questions.

Adapter

Purpose:

Make incompatible interfaces compatible.

PaymentService



PaymentGateway



AbaPaymentAdapter



ABA SDK

Adapter changes the interface.

Decorator

Purpose:

Add behavior while keeping the same interface.

PaymentGateway



LoggingDecorator



MetricsDecorator



AbaPaymentAdapter

Decorator preserves the interface.

Easy Rule

Adapter changes the interface.

Decorator keeps the interface and adds behavior.

Lesson 23 — Decorator vs Proxy

They look structurally similar.

Both may implement the same interface and wrap another object.

Example:

```
public class PaymentProxy
    implements PaymentGateway {
```

```
    private final PaymentGateway target;  
}
```

and

```
public class LoggingPaymentDecorator  
    implements PaymentGateway {  
  
    private final PaymentGateway delegate;  
}
```

So what is the difference?

Proxy

Main purpose:

Control access to the real object.

Examples:

- Authorization
- Lazy loading
- Remote access
- Caching
- Access control

Decorator

Main purpose:

Add behavior.

Examples:

- Logging
- Metrics
- Compression

- Encryption
- Retry

Comparison

Decorator	Proxy
Adds behavior	Controls access
Usually intentionally composable	Usually represents one controlled object
Many decorators may be stacked	Often one proxy layer
Focuses on enrichment	Focuses on access

Lesson 24 — Decorator vs Facade

Facade:

Client



BookingFacade

—— PaymentService

—— RoomService

—— NotificationService

Purpose:

Simplify a complex subsystem.

Decorator:

Client



LoggingDecorator



RetryDecorator



PaymentGateway

Purpose:

Add behavior to one abstraction.

Easy sentence:

Facade simplifies many components. Decorator enriches one component.

Lesson 25 — Decorator vs Strategy

Strategy:

PaymentService



PaymentStrategy

—— ABA

—— Wing

—— TrueMoney

Purpose:

Choose one algorithm.

Decorator:

Metrics



Logging



ABA

Purpose:

Add layers of behavior.

Strategy usually chooses one implementation.

Decorator can combine many wrappers.

Lesson 26 — Java I/O Is the Classic Decorator Example

This is probably the most famous Decorator example in Java.

Consider:

```
InputStream inputStream =  
    new FileInputStream("data.txt");
```

This reads raw bytes.

Now buffer it:

```
InputStream inputStream =  
    new BufferedInputStream(  
        new FileInputStream("data.txt")  
    );
```

Now add decompression:

```
InputStream inputStream =  
    new GZIPInputStream(  
        new BufferedInputStream(  
            new FileInputStream("data.gz")  
        )  
    );
```

Look at the structure:

GZIPInputStream



BufferedInputStream



FileInputStream

All are compatible with:

InputStream

That is Decorator Pattern.

Why Java I/O Works So Well

The client still uses:

```
inputStream.read();
```

It doesn't care whether the actual chain is:

FileInputStream

or:

BufferedInputStream



FileInputStream

or:

GZIPInputStream



BufferedInputStream



FileInputStream

That is exactly what we want students to recognize.

Lesson 27 — OutputStream Example

```
OutputStream outputStream =  
    new BufferedOutputStream(  
        new FileOutputStream("report.txt")  
    );
```

Another example:

```
DataOutputStream outputStream =  
    new DataOutputStream(  
        new BufferedOutputStream(  
            new FileOutputStream("data.bin")  
        )  
    );
```

Again:

DataOutputStream
↓
BufferedOutputStream
↓
FileOutputStream

Behavior is layered.

Lesson 28 — Enterprise Spring Example

Suppose we have:

```
public interface PaymentGateway {  
  
    PaymentResult pay(  
        PaymentRequest request  
    );  
}
```

Real implementation:

```
@Component  
public class ABAPaymentGateway  
    implements PaymentGateway {  
  
    @Override  
    public PaymentResult pay(  
        PaymentRequest request  
    ) {  
  
        System.out.println(  
            "Call ABA API"  
        );  
  
        return new PaymentResult(  
            "TXN-001",  
            true  
        );  
    }  
}
```

Now add enterprise behavior.

Logging Decorator

```
public class LoggingPaymentDecorator
    implements PaymentGateway {

    private final PaymentGateway delegate;

    public LoggingPaymentDecorator(
        PaymentGateway delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        System.out.println(
            "[LOG] Payment started"
        );

        PaymentResult result =
            delegate.pay(request);

        System.out.println(
            "[LOG] Payment completed"
        );

        return result;
    }
}
```

Metrics Decorator

```
public class MetricsPaymentDecorator
    implements PaymentGateway {

    private final PaymentGateway delegate;

    public MetricsPaymentDecorator(
        PaymentGateway delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        long start =
            System.nanoTime();

        try {
            return delegate.pay(request);
        } finally {

            long duration =
                System.nanoTime() - start;

            System.out.println(
                "[METRICS] duration="
                    + duration
            );
        }
    }
}
```

```
}
```

Retry Decorator

```
public class RetryPaymentDecorator
    implements PaymentGateway {

    private final PaymentGateway delegate;
    private final int maxAttempts;

    public RetryPaymentDecorator(
        PaymentGateway delegate,
        int maxAttempts
    ) {
        this.delegate = delegate;
        this.maxAttempts = maxAttempts;
    }

    @Override
    public PaymentResult pay(
        PaymentRequest request
    ) {

        RuntimeException lastException = null;

        for (int attempt = 1;
            attempt <= maxAttempts;
            attempt++) {

            try {

                System.out.println(
                    "Attempt " + attempt
                );
```

```

        return delegate.pay(request);

    } catch (RuntimeException exception) {

        lastException = exception;

    }

    throw lastException;
}
}

```

Final Composition

```

PaymentGateway gateway =
    new MetricsPaymentDecorator(
        new RetryPaymentDecorator(
            new LoggingPaymentDecorator(
                new AbaPaymentGateway()
            ),
            3
        )
    );

```

Architecture:

```

Metrics
  ↓
Retry
  ↓
Logging

```

↓
ABA

Lesson 29 — Decorator Order Matters

This is one of the most important Class 2 lessons.

Compare:

Metrics
↓
Retry
↓
Payment

with:

Retry
↓
Metrics
↓
Payment

They are not the same.

Case 1

Metrics
↓
Retry
↓
Payment

Metrics measures:

Attempt 1
+
Attempt 2
+
Attempt 3

One overall duration.

Case 2

Retry
↓
Metrics
↓
Payment

Metrics executes for every retry.

You may get:

metrics attempt 1
metrics attempt 2
metrics attempt 3

This can be useful if you want per-attempt measurement.

Another Example — Encryption and Logging

Logging

↓

Encryption

Sender

may log plaintext.

But:

Encryption

Logging

↓

Sender

may log encrypted data.

Order must be intentional.

Lesson 30 — Decorator Configuration in Spring

Instead of constructing objects manually in business code:

```
new MetricsPaymentDecorator(
    new LoggingPaymentDecorator(
```

```
        new AbaPaymentGateway()
    )
};
```

we can configure the chain in Spring.

Example:

```
@Configuration
public class PaymentConfiguration {

    @Bean
    public PaymentGateway paymentGateway(
        AbaPaymentGateway abaPaymentGateway
    ) {

        return new MetricsPaymentDecorator(
            new LoggingPaymentDecorator(
                abaPaymentGateway
            )
        );
    }
}
```

Then:

```
@Service
public class PaymentService {

    private final PaymentGateway paymentGateway;

    public PaymentService(
        PaymentGateway paymentGateway
    ) {
        this.paymentGateway = paymentGateway;
    }
}
```

```
}  
}
```

The service doesn't know the decoration structure.

Lesson 31 — Factory + Decorator

Factory and Decorator work beautifully together.

Suppose:

ABA
TrueMoney
Wing

Each provider may need different decorators.

Factory:

```
public class PaymentGatewayFactory {  
  
    public PaymentGateway create(  
        String provider  
    ) {  
  
        PaymentGateway gateway =  
            switch (provider) {  
  
                case "ABA" ->  
                    new ABAPaymentAdapter();  
  
                case "STRIPE" ->  
                    new StripePaymentAdapter();  
  
            }  
    }  
}
```

```

        default ->
            throw new IllegalArgumentException();
    };

    return decorate(gateway);
}

private PaymentGateway decorate(
    PaymentGateway gateway
) {

    return new MetricsPaymentDecorator(
        new LoggingPaymentDecorator(
            gateway
        )
    );
}
}

```

Now:

Factory

↓

Choose implementation

↓

Decorator

↓

Add behaviors

Lesson 32 — Adapter + Decorator

This combination is extremely common.

Suppose:

Your Application



PaymentGateway

External SDK:

ABA SDK

Adapter:

PaymentGateway



AbaPaymentAdapter



ABA SDK

Then decorate it:

MetricsDecorator



RetryDecorator



LoggingDecorator



AbaPaymentAdapter



ABA SDK

Each pattern solves its own problem:

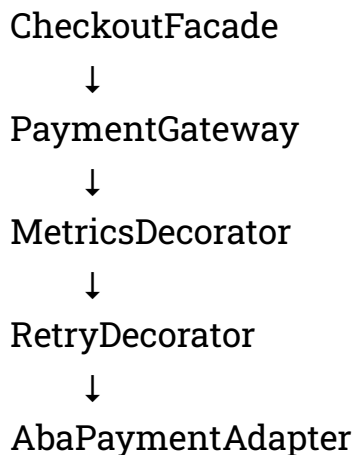
- Adapter = compatibility

- Decorator = behavior enrichment

Lesson 33 — Decorator + Facade

Also possible.

Example:



Facade coordinates checkout.

Decorator adds payment cross-cutting behavior.

Adapter integrates external provider.

Lesson 34 — Spring AOP vs Decorator

This is an important enterprise discussion.

Suppose every service needs:

- Logging
- Metrics
- Authorization
- Transaction management

We could build decorators everywhere.

But Spring already offers AOP and infrastructure mechanisms.

Example:

```
@Transactional
public void createBooking() {
}
```

Transaction behavior is added around the method.

Conceptually similar to decorating behavior.

Logging with AOP

```
@Aspect
@Component
public class LoggingAspect {

    @Around("@annotation(Logged)")
    public Object log(
        ProceedingJoinPoint joinPoint
    ) throws Throwable {

        System.out.println("Before");

        Object result =
            joinPoint.proceed();

        System.out.println("After");

        return result;
    }
}
```

This adds behavior around method execution.

When to Prefer Decorator

Use Decorator when:

- Behavior belongs to one abstraction.
- You want explicit object composition.
- Order matters.
- Different instances need different combinations.
- You want framework-independent code.

When to Prefer AOP

Use AOP when behavior applies broadly across many unrelated classes.

Examples:

Logging

Transactions

Security

Tracing

Audit

Especially when manually wrapping every object would create too much boilerplate.

Lesson 35 — WebClient Decorator-Style Wrapping

Imagine your system owns this abstraction:

```
public interface HotelClient {
```

```

        Mono<HotelResponse> getHotel(
            String hotelId
        );
    }

```

Real implementation:

```

public class WebClientHotelClient
    implements HotelClient {

    private final WebClient webClient;

    public WebClientHotelClient(
        WebClient webClient
    ) {
        this.webClient = webClient;
    }

    @Override
    public Mono<HotelResponse> getHotel(
        String hotelId
    ) {

        return webClient
            .get()
            .uri("/hotels/{id}", hotelId)
            .retrieve()
            .bodyToMono(
                HotelResponse.class
            );
    }
}

```

Add metrics:

```

public class MetricsHotelClientDecorator
    implements HotelClient {

    private final HotelClient delegate;

    public MetricsHotelClientDecorator(
        HotelClient delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public Mono<HotelResponse> getHotel(
        String hotelId
    ) {

        long start =
            System.nanoTime();

        return delegate
            .getHotel(hotelId)
            .doFinally(signal -> {

                long duration =
                    System.nanoTime()
                    - start;

                System.out.println(
                    "Duration: "
                    + duration
                );
            });
    }
}

```

Decorator works perfectly with reactive APIs too.

Lesson 36 — Caching Decorator

Another strong enterprise example.

```
public class CachingHotelClientDecorator
    implements HotelClient {

    private final HotelClient delegate;
    private final Map<String, HotelResponse> cache =
        new ConcurrentHashMap<>();

    public CachingHotelClientDecorator(
        HotelClient delegate
    ) {
        this.delegate = delegate;
    }

    @Override
    public Mono<HotelResponse> getHotel(
        String hotelId
    ) {

        HotelResponse cached =
            cache.get(hotelId);

        if (cached != null) {
            return Mono.just(cached);
        }

        return delegate
            .getHotel(hotelId)
            .doOnNext(response ->
                cache.put(
                    hotelId,
                    response
                )
            )
    }
}
```

```

        );
    }
}

```

Composition:

```

Metrics
  ↓
Caching
  ↓
WebClientHotelClient

```

Lesson 37 — Testing Decorators

Decorators should be easy to unit test.

Suppose:

```

public class LoggingPaymentDecorator
    implements PaymentGateway {
}

```

Test with a fake delegate:

```

@Test
void shouldDelegatePayment() {

    PaymentGateway delegate =
        request ->
            new PaymentResult(
                "TXN-1",
                true
            )
}

```

```
);
```

```
PaymentGateway decorator =  
    new LoggingPaymentDecorator(  
        delegate  
    );
```

```
PaymentResult result =  
    decorator.pay(  
        new PaymentRequest(...)  
    );
```

```
assertTrue(  
    result.successful()  
);  
}
```

The important thing is:

Test the decorator's responsibility, not the entire chain every time.

Retry Decorator Test

```
@Test  
void shouldRetryUntilSuccess() {  
  
    AtomicInteger attempts =  
        new AtomicInteger();  
  
    PaymentGateway unstableGateway =  
        request -> {  
  
        if (attempts.incrementAndGet() < 3) {  
            throw new RuntimeException(  
                "Temporary failure"  
            );  
        }  
    };  
}
```

```

        );
    }

    return new PaymentResult(
        "TXN-1",
        true
    );
};

PaymentGateway gateway =
    new RetryPaymentDecorator(
        unstableGateway,
        3
    );

PaymentResult result =
    gateway.pay(
        new PaymentRequest(...)
    );

assertTrue(
    result.successful()
);

assertEquals(
    3,
    attempts.get()
);
}

```

Lesson 38 — Common Enterprise Mistakes

Mistake 1 — Too Many Decorators

Imagine:

Metrics
↓
Logging
↓
Audit
↓
Retry
↓
Tracing
↓
Caching
↓
Security
↓
RateLimit
↓
Validation
↓
Encryption
↓
Payment

At some point, understanding execution becomes difficult.

Use Decorator carefully.

Mistake 2 — Hidden Order Dependencies

If:

A must always wrap B

and nobody knows this, bugs happen.

Document important order rules.

Mistake 3 — Decorators Knowing Each Other

Bad:

```
if (delegate
    instanceof LoggingDecorator) {
}
```

Decorators should depend only on the shared abstraction.

Mistake 4 — Decorator Changes the Contract

Suppose:

PaymentGateway.pay(...)

normally throws:

PaymentFailedException

but one decorator silently returns null.

That breaks substitutability.

Decorators should preserve the behavioral contract.

This connects to LSP.

Mistake 5 — Using Decorator for Workflow

If you're coordinating:

Validate

Reserve
Pay
Create Booking
Notify

that's not Decorator.

That sounds more like:

Facade
Template Method
Application Service

Decorator is for adding behavior around one abstraction.

Lesson 39 — SOLID Deeper Connection

OCP

Add new decorator:

TracingPaymentDecorator

Existing payment implementations remain unchanged.

SRP

Each decorator has one cross-cutting responsibility.

DIP

Clients depend on abstraction:

PaymentGateway

LSP

Every decorator should behave as a valid:

PaymentGateway

Clients should not care whether they receive:

AbaPaymentAdapter

or:

Metrics

↓

Logging

↓

AbaPaymentAdapter

Both must respect the same contract.

Homework Answer — Storage Decorator

Target:

```
public interface StorageService {  
  
    void upload(  
        String filename,  
        byte[] content  
    );  
  
    byte[] download(  
        String filename
```

```
);  
}
```

Concrete Component

```
public class LocalStorageService  
    implements StorageService {  
  
    private final Map<String, byte[]> storage =  
        new HashMap<>();  
  
    @Override  
    public void upload(  
        String filename,  
        byte[] content  
    ) {  
  
        storage.put(  
            filename,  
            content  
        );  
    }  
  
    @Override  
    public byte[] download(  
        String filename  
    ) {  
  
        return storage.get(filename);  
    }  
}
```

Base Decorator

```
public abstract class StorageDecorator
    implements StorageService {

    protected final StorageService delegate;

    protected StorageDecorator(
        StorageService delegate
    ) {
        this.delegate = delegate;
    }
}
```

Logging Decorator

```
public class LoggingStorageDecorator
    extends StorageDecorator {

    public LoggingStorageDecorator(
        StorageService delegate
    ) {
        super(delegate);
    }

    @Override
    public void upload(
        String filename,
        byte[] content
    ) {

        System.out.println(
            "Upload: " + filename
        );
    }
}
```

```

        delegate.upload(
            filename,
            content
        );
    }

    @Override
    public byte[] download(
        String filename
    ) {

        System.out.println(
            "Download: " + filename
        );

        return delegate.download(
            filename
        );
    }
}

```

Encryption Decorator

```

public class EncryptionStorageDecorator
    extends StorageDecorator {

    public EncryptionStorageDecorator(
        StorageService delegate
    ) {
        super(delegate);
    }

    @Override

```

```

public void upload(
    String filename,
    byte[] content
) {

    byte[] encrypted =
        encrypt(content);

    delegate.upload(
        filename,
        encrypted
    );
}

@Override
public byte[] download(
    String filename
) {

    byte[] encrypted =
        delegate.download(filename);

    return decrypt(encrypted);
}

private byte[] encrypt(
    byte[] content
) {

    byte[] result =
        content.clone();

    for (int i = 0;
        i < result.length;
        i++) {

        result[i] =
            (byte) (

```

```

        result[i] ^ 1
    );
}

return result;
}

private byte[] decrypt(
    byte[] content
) {
    return encrypt(content);
}
}

```

Compression Decorator

For classroom simplicity:

```

public class CompressionStorageDecorator
    extends StorageDecorator {

    public CompressionStorageDecorator(
        StorageService delegate
    ) {
        super(delegate);
    }

    @Override
    public void upload(
        String filename,
        byte[] content
    ) {

        System.out.println(
            "Compress before upload"

```

```

        );

        delegate.upload(
            filename,
            content
        );
    }

    @Override
    public byte[] download(
        String filename
    ) {

        byte[] content =
            delegate.download(filename);

        System.out.println(
            "Decompress after download"
        );

        return content;
    }
}

```

In the runnable sample project, I would use real GZIP compression so students can observe actual byte-size changes.

Metrics Decorator

```

public class MetricsStorageDecorator
    extends StorageDecorator {

    public MetricsStorageDecorator(
        StorageService delegate
    ) {

```

```

        super(delegate);
    }

    @Override
    public void upload(
        String filename,
        byte[] content
    ) {

        long start =
            System.nanoTime();

        try {

            delegate.upload(
                filename,
                content
            );

        } finally {

            System.out.println(
                "Upload duration: "
                    + (
                        System.nanoTime()
                            - start
                    )
            );
        }
    }

    @Override
    public byte[] download(
        String filename
    ) {

        long start =
            System.nanoTime();

```

```

try {

    return delegate.download(
        filename
    );

} finally {

    System.out.println(
        "Download duration: "
        + (
            System.nanoTime()
            - start
        )
    );
}
}
}

```

Final Storage Chain

```

StorageService storage =
    new MetricsStorageDecorator(
        new LoggingStorageDecorator(
            new EncryptionStorageDecorator(
                new CompressionStorageDecorator(
                    new LocalStorageService()
                )
            )
        )
    );

```

Execution:

Metrics



Logging



Encryption



Compression



Local Storage

Download returns through the chain in reverse:

Local Storage



Decompression



Decryption



Logging



Metrics

That reverse flow is a very important teaching point.

Bonus — Decorate S3 Adapter

From the previous Adapter lesson:

```
StorageService storage =  
    new S3StorageAdapter(  
        new AmazonS3Client()
```

```
);
```

Now:

```
storage =  
    new MetricsStorageDecorator(  
        new LoggingStorageDecorator(  
            new EncryptionStorageDecorator(  
                storage  
            )  
        )  
    );
```

Architecture:

```
Metrics Decorator  
  ↓  
Logging Decorator  
  ↓  
Encryption Decorator  
  ↓  
S3 Storage Adapter  
  ↓  
AWS SDK
```

This is a beautiful combination of patterns.

Interview Questions and Answers

1. What problem does Decorator Pattern solve?

It adds optional responsibilities to an object dynamically without modifying the original implementation or creating a large inheritance hierarchy.

2. Why use Decorator instead of inheritance?

Inheritance creates static combinations and can lead to subclass explosion.

Decorator uses composition, allowing behaviors to be combined dynamically.

3. What are the main participants?

- Component
- Concrete Component
- Decorator
- Concrete Decorator

4. Why does Decorator implement the same interface?

So the decorated object can be used wherever the original component is expected.

This enables transparent substitution and decorator chaining.

5. What does Decorator contain?

Usually another object of the same abstraction:

```
private final PaymentGateway delegate;
```

6. Can decorators be combined?

Yes.

That's one of the main advantages.

7. Does decorator order matter?

Yes.

For example:

Metrics → Retry → Payment

and:

Retry → Metrics → Payment

measure different things.

8. Decorator vs Adapter?

Adapter changes interface.

Decorator keeps interface and adds behavior.

9. Decorator vs Proxy?

Decorator enriches behavior.

Proxy controls access.

10. Decorator vs Strategy?

Strategy chooses one behavior.

Decorator layers multiple behaviors.

11. Give a Java example.

Java I/O:

```
new BufferedInputStream(  
    new FileInputStream(file)  
);
```

12. Can Decorator be used in Spring?

Yes.

Decorators can be configured as beans or created through factory/configuration classes.

13. Can Decorator work with reactive programming?

Yes.

A decorator can return Mono or Flux and use operators such as:

```
doOnNext()  
doOnError()  
doFinally()  
retryWhen()
```

14. When should AOP be preferred?

When the same cross-cutting concern applies broadly across many unrelated classes and explicit wrapping would add unnecessary boilerplate.

15. What is a disadvantage of Decorator?

Large decorator chains can become difficult to understand, debug, configure, and order correctly.

Final Pattern Comparison

Pattern	Main Purpose
Strategy	Choose behavior
Factory	Choose/create object
Builder	Construct complex object
Template Method	Define fixed workflow
Observer	Notify interested components
Adapter	Make incompatible interfaces compatible
Facade	Simplify a complex subsystem
Decorator	Add behavior while keeping the same interface

Decorator wraps the same abstraction, adds one responsibility, and delegates the original work.